Experiment-13

Write A Python Program To Implement Minimax Algorithm For Gaming

Aim

To write and execute a Python program to implement the Minimax Algorithm for the Tic-Tac-Toe game, enabling the computer to select the best possible move.

Algorithm

1. Start the program.
2. Initialize a 3 × 3 game board with empty spaces.
3. Assign X to the human player and O to the computer.
4. Display the game board.
5. Get the human player's move.
6. Check whether the move is valid and update the board.
7. Check whether the human player has won or the game is a draw.
8. Generate all possible moves for the computer.
9. Apply the Minimax algorithm to evaluate each possible move.
10. Select the move with the highest score for the computer.
11. Repeat the process until a player wins or the game ends in a draw.
12. Display the final result and stop the program.

Code

```
# Minimax Algorithm for Tic-Tac-Toe

board = [" "] * 9

def display():
    print()
    for i in range(0, 9, 3):
        print(" | ".join(board[i:i+3]))
        if i < 6:
            print("--+---+--")
    print()

def winner(b):
    wins = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),
        (0, 3, 6), (1, 4, 7), (2, 5, 8),
        (0, 4, 8), (2, 4, 6)
    ]
    for a, c, d in wins:
        if b[a] == b[c] == b[d] != " ":
            return b[a]
    return None

def minimax(b, is_max):
    result = winner(b)
    if result == "O":
        return 1
    if result == "X":
        return -1
    if " " not in b:
        return 0

    if is_max:
        best = -float("inf")
        for i in range(9):
            if b[i] == " ":
                b[i] = "O"
                score = minimax(b, False)
                b[i] = " "
                best = max(best, score)
        return best
    else:
        best = float("inf")
        for i in range(9):
            if b[i] == " ":
                b[i] = "X"
                score = minimax(b, True)
```

```
            board[i] = "O"
            score = minimax(board, False)
            board[i] = " "

            if score > best_score:
                best_score = score
                move = i

    if move is not None:
        board[move] = "O"


def play():
    print("TIC-TAC-TOE USING MINIMAX")
    print("You are X; Computer is O.")

    while True:
        display()

        try:
            pos = int(input("Enter position (1-9): ")) - 1
        except ValueError:
            print("Enter a valid number.")
            continue

        if pos < 0 or pos > 8 or board[pos] != " ":
            print("Invalid move. Try again.")
            continue

        board[pos] = "X"

        if winner(board) == "X":
            display()
            print("You win!")
            break

        if " " not in board:
            display()
            print("It's a draw!")
            break

        computer_move()

        if winner(board) == "O":
            display()
            print("Computer wins!")
            break

        if " " not in board:
            display()
            print("It's a draw!")
            break


play()
```

Output

```
Python 3.10.0 (tags/v3.10.0:b494f59, Oct  4 2021, 19:00:18) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
============================== RESTART: D:/ex-13.py ==============================
TIC-TAC-TOE USING MINIMAX
You are X; Computer is O.

   |   |
--+---+--
   |   |
--+---+--
   |   |

Enter position (1-9): 1

X |   |
--+---+--
  | O |
--+---+--
   |   |

Enter position (1-9):
```

Result

The Minimax Algorithm was successfully implemented for the Tic-Tac-Toe game. The computer evaluates possible moves and selects the best move to maximize its chances of winning or achieving a draw.